



UNAPEC

UNIVERSIDAD APEC

Integrantes

Bryan Candelario Pérez - A00115174

Sara Valdez Bautista - A00115069

Carlos Rafael Torres - A00115074

Darwin Muñoz Román - A00114905

Período académico

Mayo - Agosto 2026 (C2)

Fecha de Entrega

15 de agosto de 2026

Profesor

Huygens Brito

Materia

ISO905 - Arquitectura de Desarrollo con Tecnología Propietaria

Tema de estudio

TRABAJO SAD

Tabla de contenido

1. Introducción	1
1.1. Propósito.....	1
1.2. Alcance.....	2
1.3. Definiciones, acrónimos y abreviaturas.....	3
1.4 Referencias.....	4
1.4.1 Descripción de la Metodología usada.....	5
1.5 Visión general del documento	5
2. Representación Arquitectónica	7
2.1. Selección de Arquitecturas según los Atributos de Calidad	7
2.2. Representación arquitectónica	8
2.2.1. Clientes pesados	9
2.2.1.1. Presentación	9
2.2.1.1.1. Modelo Vista Controlador	10
2.2.1.2. Seguridad.....	11
2.2.1.3. Lógica del Negocio (Core).....	12
2.2.2. Servidor.....	12
2.2.3. Capas.....	13
2.3. Representación arquitectónica	15
3. Objetivos y Restricciones Arquitectónicas	16
3.1. Priorización de requerimientos	16
3.1.1. Resumen de ponderación por Stakeholder	16

3.1.2. Resumen de requerimientos.....	17
3.1.3. Requerimientos de prioridad mayor o igual a 8	18
3.2. Metas y restricciones arquitectónicas según requerimientos	19
4. Vista de Casos de Uso	20
5. Vista Lógica.....	25
5.1. Visión General	26
5.2. Diseño Arquitectónico de Paquetes Importantes.....	26
5.2.1. Paquete Presentación.....	26
5.2.2. Lógica del Negocio.....	26
5.2.3. Conexión.....	27
5.2.4. Datos	27
6. Vista de Procesos	27
7. Vista de Despliegue.....	28
8. Vista de Implementación.....	29
8.1. Visión General	29
8.2. Capas	29
8.2.1. Presentación	29
8.2.2. Seguridad.....	30
8.2.3. Lógica de Negocio (Core)	31
8.2.4. Datos	31
9. Vista de Datos.....	32
10. Tamaño y Desempeño	34

10.1. Concurrencia de usuarios	34
11. Calidad	34
11.1. Seguridad	34
11.1.1. Despliegue de la información	34
11.1.2. Validación de la información del usuario	35
11.2. Usabilidad	35
11.2.1. Interfaz intuitiva y mapas interactivos	35
11.2.2. Ayudas tipo ToolTipText	35

1. Introducción

La presente sección tiene como propósito proporcionar una visión general del Documento de Arquitectura de Software (Software Architecture Document - SAD) del sistema **Accelerilate**, desarrollado para Autos Marbella. En este documento se describe la arquitectura propuesta para el sistema, definiendo la estructura de sus componentes, las tecnologías involucradas, las decisiones de diseño arquitectónico y la forma en que estas satisfacen los requerimientos funcionales y no funcionales establecidos en la Especificación de Requerimientos de Software (SRS).

Asimismo, el documento sirve como guía para los diferentes participantes del proyecto durante las etapas de diseño, desarrollo, pruebas, despliegue y mantenimiento del sistema, asegurando una comprensión común de la solución arquitectónica seleccionada.

1.1. Propósito

El propósito de este documento es definir la arquitectura de software que será utilizada para el desarrollo del sistema **Accelerilate**, una plataforma orientada a optimizar la gestión de alquiler de vehículos de **Autos Marbella**.

En este documento se describen las decisiones arquitectónicas adoptadas para satisfacer los requerimientos funcionales y no funcionales definidos en el Documento de Especificación de Requerimientos de Software (SRS), estableciendo la organización de los componentes, las tecnologías seleccionadas, los mecanismos de comunicación entre módulos y las interfaces con sistemas externos.

El SAD está dirigido a todos los **stakeholders** involucrados en el proyecto, entre ellos:

Gerente del proyecto.

Analista de requerimientos.

Arquitecto de software.

Equipo de desarrollo.

Equipo de pruebas (QA).

Administradores del sistema.

Personal técnico de Autos Marbella.

Este documento constituye la principal referencia técnica para la implementación del sistema, permitiendo mantener la consistencia del diseño durante todo el ciclo de vida del software y facilitando futuras actividades de mantenimiento, evolución e integración con nuevos servicios.

1.2. Alcance

El presente Documento de Arquitectura de Software describe la solución arquitectónica propuesta para el sistema Accelerilate, desarrollado para la empresa Autos Marbella, cuyo objetivo principal es automatizar y centralizar la gestión de alquiler de vehículos.

La arquitectura contempla los componentes necesarios para soportar las funcionalidades definidas en el SRS, entre las cuales se encuentran:

- Administración de reservas de vehículos.
- Gestión de clientes.
- Administración de la flota vehicular.
- Control de mantenimiento preventivo y correctivo.
- Gestión de pagos.
- Administración de usuarios y roles.
- Generación de reportes operativos y administrativos.
- Monitoreo de vehículos mediante integración con dispositivos GPS.

Además, este documento describe la organización lógica del sistema, la distribución de sus componentes, la arquitectura de despliegue, el modelo de datos, las interfaces de comunicación y las tecnologías utilizadas para garantizar atributos de calidad como:

- Escalabilidad.
- Disponibilidad.
- Seguridad.
- Mantenibilidad.
- Rendimiento.
- Confiabilidad.

También se especifica la forma en que el sistema interactúa con servicios externos, tales como plataformas de pago electrónico, sistemas de facturación y dispositivos GPS, así como las restricciones técnicas y de infraestructura consideradas durante el diseño arquitectónico.

1.3. Definiciones, acrónimos y abreviaturas

Los términos técnicos, siglas y abreviaturas utilizados en este documento corresponden a conceptos ampliamente empleados en el área de arquitectura y desarrollo de software. Con el fin de evitar ambigüedades y facilitar la comprensión del documento, a continuación, se presentan los principales términos utilizados.

SAD: Software Architecture Document, documento que describe la arquitectura del sistema.

SRS: Software Requirements Specification, Documento de especificación de requerimientos del software.

API: Application Programming Interface. Conjunto de servicios que permiten la comunicación entre aplicaciones.

REST: Representational State Transfer. Estilo arquitectónico para el desarrollo de servicios web.

GPS: Global Positioning System. Sistema utilizado para la localización geográfica de los vehículos.

UI: User Interface. Interfaz mediante la cual el usuario interactúa con el sistema.

BD: Base de Datos. Sistema encargado del almacenamiento de la información del software.

HTTPS: HyperText Transfer Protocol Secure. Protocolo seguro para la transmisión de información mediante Internet.

CRUD: Create, Read, Update and Delete. Operaciones básicas sobre los datos del sistema.

Stakeholders: Personas u organizaciones interesadas en el desarrollo y funcionamiento del sistema.

Los términos adicionales que no se encuentren definidos en esta sección deberán interpretarse de acuerdo con el glosario incluido en la documentación técnica del proyecto y el Documento de Especificación de Requerimientos de Software (SRS).

1.4 Referencias

El desarrollo de la arquitectura de **Accelerilate** se fundamenta en estándares internacionales de ingeniería de software, patrones de arquitectura, documentación técnica y especificaciones utilizadas durante el análisis y diseño del sistema.

Entre las principales referencias utilizadas se encuentran:

- Documento de Especificación de Requerimientos de Software (SRS) del proyecto Accelerilate.
- IEEE Std 1016™-2022 — *IEEE Standard for Information Technology—Systems Design—Software Design Description*.
- ISO/IEC/IEEE 42010:2022 — *Architecture Description*.
- Sommerville, I. (2021). *Software Engineering* (11th ed.).
- Bass, L., Clements, P., & Kazman, R. (2022). *Software Architecture in Practice* (4th ed.).
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*.
- Oracle. *Java Platform Documentation*.
- Spring Framework Documentation.
- MySQL Reference Manual.

1.4.1 Descripción de la Metodología usada

Para el desarrollo de la arquitectura de software del sistema Accelerilate se empleó una metodología basada en las recomendaciones establecidas por el estándar ISO/IEC/IEEE 42010, complementada con la estructura propuesta por el estándar IEEE 1016 para la documentación de arquitecturas de software.

El proceso arquitectónico parte del análisis de los requerimientos funcionales y no funcionales definidos en el Documento de Especificación de Requerimientos de Software (SRS). A partir de estos requerimientos se identifican los componentes principales del sistema, sus responsabilidades, las relaciones entre ellos y los mecanismos de comunicación necesarios para satisfacer los objetivos del negocio.

La arquitectura será documentada utilizando el modelo de vistas 4+1 de Kruchten, permitiendo representar el sistema desde diferentes perspectivas:

- Vista de Casos de Uso.
- Vista Lógica.
- Vista de Procesos.
- Vista de Desarrollo (Implementación).
- Vista Física (Despliegue).

Este enfoque facilita la comunicación entre los diferentes participantes del proyecto y permite evaluar aspectos relacionados con escalabilidad, rendimiento, seguridad, mantenibilidad y disponibilidad antes de la implementación del sistema.

1.5 Visión general del documento

El presente Documento de Arquitectura de Software está organizado en diferentes secciones que describen la solución arquitectónica propuesta para el sistema **Accelerilate**.

La **Sección 2. Representación Arquitectónica** presenta la arquitectura general del sistema, el estilo arquitectónico seleccionado y la forma en que se representan sus componentes principales.

La **Sección 3. Objetivos y Restricciones Arquitectónicas** describe los requerimientos no funcionales que influyen en el diseño de la arquitectura, incluyendo aspectos relacionados con seguridad, rendimiento, disponibilidad, mantenibilidad y escalabilidad.

La **Sección 4. Vista de Casos de Uso** presenta la interacción entre los actores y las funcionalidades principales del sistema.

La **Sección 5. Vista Lógica** describe la organización de los módulos del sistema, las relaciones entre componentes y la estructura interna de la solución.

La **Sección 6. Vista de Procesos** explica el comportamiento dinámico del sistema, los procesos ejecutados y la interacción entre los distintos servicios durante la operación.

La **Sección 7. Vista de Despliegue** muestra la infraestructura física y lógica donde será instalado el sistema, incluyendo servidores, base de datos, dispositivos clientes y servicios externos.

La **Sección 8. Vista de Implementación** describe la organización del código fuente, paquetes, componentes de software y tecnologías utilizadas durante el desarrollo.

La **Sección 9. Vista de Datos** presenta el modelo de almacenamiento de la información, las principales entidades del sistema y sus relaciones.

La **Sección 10. Tamaño y Desempeño** describe las consideraciones relacionadas con capacidad, tiempos de respuesta, concurrencia y uso de recursos.

Finalmente, la **Sección 11. Calidad** expone los atributos de calidad que sustentan la arquitectura del sistema, tales como seguridad, confiabilidad, disponibilidad, escalabilidad, mantenibilidad y usabilidad, justificando las decisiones arquitectónicas adoptadas para satisfacer las necesidades operativas de Autos Marbella.

2. Representación Arquitectónica

En esta sección se describe la arquitectura de software seleccionada para el sistema **Accelerilate**, justificando la elección del estilo arquitectónico con base en los atributos de calidad definidos para el proyecto. Asimismo, se presentan los principales componentes de la arquitectura, su organización en capas y la forma en que interactúan entre sí para satisfacer los requerimientos funcionales y no funcionales establecidos en el Documento de Especificación de Requerimientos de Software (SRS).

La arquitectura propuesta busca proporcionar una solución escalable, segura, mantenible y de alto rendimiento, permitiendo la administración eficiente de las reservas, la gestión de clientes, el control de la flota vehicular, el procesamiento de pagos y el monitoreo de los vehículos mediante integración con dispositivos GPS.

2.1. Selección de Arquitecturas según los Atributos de Calidad

La selección de la arquitectura para **Accelerilate** se realizó considerando los atributos de calidad definidos durante el análisis de los requerimientos del sistema.

Para ello se analizaron distintos estilos arquitectónicos ampliamente utilizados en aplicaciones empresariales, entre ellos:

- Arquitectura Monolítica
- Arquitectura Cliente-Servidor
- Arquitectura por Capas
- Arquitectura Basada en Servicios (SOA)
- Arquitectura de Microservicios

Cada una de estas alternativas fue evaluada con respecto a los atributos de calidad considerados críticos para el proyecto.

Los atributos de calidad priorizados fueron los siguientes:

Categoría	Subcategorías
Funcionalidad	Idoneidad, Seguridad, Interoperabilidad
Fiabilidad	Disponibilidad, Recuperación ante fallos

Rendimiento	Tiempo de respuesta, Uso eficiente de recursos
Mantenibilidad	Modularidad, Facilidad de pruebas, Facilidad de cambio
Escalabilidad	Crecimiento horizontal y vertical
Usabilidad	Facilidad de aprendizaje, Accesibilidad

Después del análisis comparativo, se determinó que la arquitectura que mejor satisface las necesidades del proyecto corresponde a una Arquitectura Cliente-Servidor organizada mediante una Arquitectura por Capas.

Esta decisión se fundamenta en que el sistema será utilizado simultáneamente por múltiples usuarios (administradores, empleados y clientes), quienes accederán a una única fuente centralizada de información mediante una aplicación web. Además, la separación en capas facilita el mantenimiento, la reutilización del código, las pruebas unitarias y la incorporación futura de nuevas funcionalidades.

La arquitectura seleccionada también facilita la integración con servicios externos como plataformas de pago electrónico, sistemas de facturación y dispositivos GPS, manteniendo un bajo acoplamiento entre los distintos componentes del sistema.

2.2. Representación arquitectónica

La arquitectura de **Accelerilate** está basada en un modelo **Cliente-Servidor en Capas**, donde la aplicación cliente se comunica mediante servicios REST con un servidor central encargado del procesamiento de la lógica de negocio y del acceso a la base de datos.

La comunicación entre cliente y servidor se realiza mediante el protocolo HTTPS, garantizando la confidencialidad e integridad de la información transmitida.

La arquitectura está compuesta por los siguientes componentes principales:

- Cliente Web
- API REST
- Servicios de Negocio
- Base de Datos
- Servicios Externos

- Dispositivos GPS

Esta organización permite separar claramente las responsabilidades del sistema, reduciendo el acoplamiento entre componentes y facilitando su evolución.

2.2.1. Clientes pesados

El cliente corresponde a una aplicación web responsiva que será utilizada desde navegadores modernos tanto en computadoras de escritorio como en dispositivos móviles.

Esta aplicación será la encargada de presentar la información al usuario, validar los datos básicos ingresados y consumir los servicios expuestos por el servidor.

Los usuarios del sistema podrán acceder a funcionalidades como:

- Gestión de reservas.
- Administración de vehículos.
- Gestión de clientes.
- Registro de pagos.
- Consulta del estado de la flota.
- Administración del mantenimiento.
- Generación de reportes.

El cliente estará dividido en tres capas principales.

2.2.1.1. Presentación

La capa de presentación constituye la interfaz entre el usuario y el sistema.

Su responsabilidad principal consiste en mostrar la información de forma clara e intuitiva, permitiendo que los usuarios interactúen con las diferentes funcionalidades del sistema.

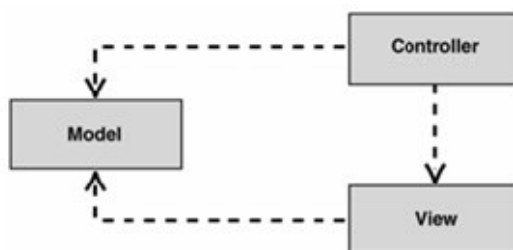
La interfaz será desarrollada bajo un modelo basado en componentes, permitiendo reutilizar elementos de la interfaz y mantener una experiencia uniforme para todos los módulos del sistema.

Entre sus responsabilidades se encuentran:

- Visualización de información.
- Captura de datos.
- Navegación entre módulos.
- Validaciones básicas del lado del cliente.
- Comunicación con los servicios REST.

2.2.1.1.1. Modelo Vista Controlador

El patrón de diseño Modelo-Vista-Controlador (MVC) organiza un sistema en tres componentes principales que trabajan de manera coordinada, pero con responsabilidades separadas. El modelo gestiona los datos y el comportamiento del sistema, respondiendo tanto a solicitudes de información como a instrucciones de cambio de estado. La vista se ocupa de la representación visual de esos datos, mostrando la información al usuario de distintas formas sin afectar la lógica interna.



Tanto la vista como el controlador dependen del modelo, el cual no depende de las otras clases. Esta separación permite construir y probar el modelo independientemente de la

representación visual, permitiendo cambios en el controlador y la vista sin afectar la funcionalidad del sistema.

Entre las ventajas del estilo señaladas en la documentación de Patterns & Practices de Microsoft están las siguientes:

Soporte de vistas múltiples: dado que la vista se halla separada del modelo y no hay dependencia directa del modelo con respecto a la vista, la interfaz de usuario puede mostrar múltiples vistas de los mismos datos simultáneamente. Por ejemplo, múltiples páginas de una aplicación de Web pueden utilizar el mismo modelo de objetos, mostrado de maneras diferentes.

Adaptación al cambio: los requerimientos de interfaz de usuario tienden a cambiar con mayor rapidez que las reglas de negocios. Los usuarios pueden preferir distintas opciones de representación, o requerir soporte para nuevos dispositivos como teléfonos celulares o PDAs. Dado que el modelo no depende de las vistas, agregar nuevas opciones de presentación generalmente no afecta al modelo.

2.2.1.2. Seguridad

La capa de seguridad será responsable de controlar el acceso al sistema mediante mecanismos de autenticación y autorización.

Entre sus funciones se encuentran:

- Inicio de sesión.
- Validación de credenciales.
- Administración de sesiones.
- Control de permisos según roles.
- Protección de rutas privadas.

- Cifrado de las comunicaciones mediante HTTPS.

El sistema implementará diferentes perfiles de usuario, garantizando que cada uno únicamente pueda acceder a las funcionalidades correspondientes a sus responsabilidades.

2.2.1.3. Lógica del Negocio (Core)

La lógica de negocio ubicada en el cliente estará orientada principalmente al manejo del estado de la interfaz y la validación de datos antes de ser enviados al servidor.

Entre sus responsabilidades se encuentran:

- Validaciones de formularios.
- Gestión del estado de la aplicación.
- Consumo de servicios REST.
- Manejo de errores.
- Presentación de mensajes al usuario.

Las reglas críticas del negocio permanecerán centralizadas en el servidor para garantizar la integridad de la información.

2.2.2. Servidor

El servidor constituye el núcleo del sistema **Accelerilate**.

Es el encargado de ejecutar todas las reglas de negocio relacionadas con:

- Gestión de reservas.
- Administración de clientes.
- Gestión de vehículos.
- Procesamiento de pagos.
- Control de mantenimiento.
- Gestión de usuarios.
- Generación de reportes.
- Integración con servicios externos.

Asimismo, administra el acceso a la base de datos y controla las comunicaciones con plataformas de pago y dispositivos GPS.

Toda la información procesada por el sistema se almacena de manera centralizada, garantizando la consistencia y disponibilidad de los datos.

2.2.3. Capas

La arquitectura de Accelerilate se encuentra organizada en cuatro capas principales.

Capa de Presentación: es responsable de la interacción entre los usuarios y el sistema. Esta proporciona una interfaz gráfica intuitiva que permite acceder a las funcionalidades del software desde navegadores web modernos.

Capa de Aplicación: esta capa contiene los controladores y servicios encargados de coordinar el flujo de información entre la interfaz de usuario y la lógica de negocio. Sus principales responsabilidades incluyen:

- Recepción de solicitudes HTTP.
- Validación de datos.
- Coordinación de servicios.
- Exposición de APIs REST.

Capa de Dominio (Negocio): representa el núcleo funcional del sistema. Aquí se implementan todas las reglas del negocio relacionadas con:

- Disponibilidad de vehículos.
- Gestión de reservas.
- Cálculo de tarifas.
- Procesamiento de pagos.
- Mantenimiento de vehículos.
- Administración de clientes.
- Gestión de reportes.

La separación del dominio permite mantener un bajo acoplamiento entre las reglas del negocio y las tecnologías utilizadas.

Capa de Datos: es la encargada de almacenar y recuperar la información del sistema. Se comunica con una base de datos relacional mediante mecanismos de persistencia, permitiendo administrar información como:

- Vehículos.
- Clientes.
- Usuarios.
- Reservas.
- Pagos.
- Facturas.
- Mantenimientos.
- Historial de ubicaciones GPS.

Esta capa garantiza la integridad, consistencia y disponibilidad de la información.

Beneficios de la arquitectura por capas

La arquitectura seleccionada proporciona múltiples ventajas para el proyecto, entre ellas:

- Separación clara de responsabilidades.
- Mayor facilidad de mantenimiento.
- Reutilización de componentes.
- Escalabilidad del sistema.
- Facilidad para realizar pruebas unitarias e integración.
- Bajo acoplamiento entre módulos.
- Mayor seguridad en el acceso a la información.
- Facilidad para incorporar nuevas funcionalidades sin afectar los componentes existentes.

2.3. Representación arquitectónica

Para documentar la arquitectura del sistema Accelerilate se utilizará el modelo de vistas 4+1 de Philippe Kruchten, el cual permite representar la arquitectura desde diferentes perspectivas, facilitando la comprensión por parte de desarrolladores, arquitectos, administradores del sistema y demás interesados.

Las vistas consideradas en este documento son las siguientes:

Vista de Casos de Uso: describe las funcionalidades principales del sistema desde la perspectiva de los usuarios, representando la interacción entre los actores y los servicios ofrecidos por la aplicación.

Vista Lógica: presenta la organización de los componentes internos del sistema, los módulos funcionales y las relaciones existentes entre ellos mediante diagramas UML.

Vista de Procesos: describe el comportamiento dinámico del sistema, mostrando la interacción entre procesos concurrentes, servicios y comunicaciones necesarias para soportar múltiples usuarios simultáneamente.

Vista de Despliegue: representa la infraestructura física donde será ejecutado el sistema, incluyendo servidores de aplicaciones, bases de datos, clientes web y servicios externos como plataformas de pago y dispositivos GPS.

Vista de Implementación: describe la organización del código fuente, los paquetes, componentes y dependencias del sistema, facilitando las actividades de desarrollo y mantenimiento.

Vista de Datos: presenta el modelo lógico de la base de datos, las entidades principales, sus relaciones y la estrategia utilizada para garantizar la persistencia e integridad de la información.

En conjunto, estas vistas ofrecen una representación integral de la arquitectura de Accelerilate, permitiendo comprender tanto su estructura estática como su comportamiento dinámico y su despliegue en el entorno operativo.

3. Objetivos y Restricciones Arquitectónicas

3.1. Priorización de requerimientos

El proceso de priorización de requerimientos se realizó tomando a los Stakeholders involucrados en el proyecto de **Accelerilate** para Autos Marbella, y analizando la importancia que cada uno de ellos tiene sobre cada requerimiento.

La primera parte de la priorización consiste en definir el peso de cada Stakeholder dentro del proyecto, ya que su nivel de influencia e interés varía según las áreas operativas (administración de reservas, monitoreo de flota, contabilidad, etc.). Esto se realiza cruzando los requerimientos contra los Stakeholders para identificar quiénes están interesados en cada funcionalidad. Luego, se contabiliza el número de requerimientos de interés por Stakeholder para obtener su porcentaje de participación.

Con el peso de cada Stakeholder definido, este valor se utiliza para calcular la prioridad final de cada requerimiento, sumando los pesos de los Stakeholders interesados en él.

En las siguientes subsecciones se muestra el resumen de los pesos de los Stakeholders, un resumen general de todos los requerimientos del sistema Accelerilate y aquellos con prioridad superior a 7, con el fin de justificar las decisiones arquitectónicas tomadas.

3.1.1. Resumen de ponderación por Stakeholder

La Tabla 2 muestra un resumen de la ponderación asignada a cada Stakeholder clave del sistema Accelerilate.

Stakeholder	Peso del Stakeholder
Gerente de Autos Marbella	9,5
Cliente / Usuario Final	8,8
Administrador del Sistema	8,0

Encargado de Flota/Mantenimiento	7,5
Arquitecto de Software	6,5
Analista de Requerimientos	6,0
Desarrollador	5,5
Proveedor Externo (GPS / Pagos)	4,0

Tabla 2. Resumen de ponderación por Stakeholder. *Dicha ponderación se realizó tomando en cuenta la cantidad de requerimientos y el impacto directo que cada actor tiene sobre el core del negocio (renta de vehículos).*

3.1.2. Resumen de requerimientos

La Tabla 3 muestra un resumen de los requerimientos funcionales (RF) y no funcionales (RNF) del sistema Accelerilate.

ID	Descripción	Prioridad
RF-01-01	El sistema permite al administrador gestionar (crear, modificar, cancelar) reservas de vehículos.	10
RF-01-02	El sistema centraliza y almacena en base de datos la información de clientes, vehículos y transacciones.	9
RF-02-01	El sistema se integra con 5 plataformas de pago en línea para procesar transacciones.	10
RF-03-01	El sistema recibe datos en tiempo real de los dispositivos GPS instalados en la flota.	9
RF-03-02	El sistema permite al administrador visualizar la ubicación de los vehículos en un mapa.	8
RF-04-01	El sistema genera alertas y lleva el control de mantenimiento de la flota de vehículos.	8
RF-05-01	El sistema se comunica con el software de facturación para generar reportes contables.	7

RNF-10-01	La interfaz de usuario es responsiva, accesible desde navegadores web modernos en escritorio y móviles.	10
RNF-11-01	El consumo de recursos en memoria del servidor debe ser ligero y eficiente.	9
RNF-12-01	La comunicación de datos y acceso remoto se realiza a través de internet mediante protocolos seguros (HTTPS/SSL).	10
RNF-13-01	El sistema es autónomo en su funcionamiento principal y no depende exclusivamente de sistemas externos para crear reservas.	8

Tabla 3. Resumen de requerimientos del sistema Accelerilate.

3.1.3. Requerimientos de prioridad mayor o igual a 8

La Tabla 4 muestra ordenados de mayor a menor los requerimientos según su alta prioridad para el funcionamiento crítico de Accelerilate.

ID	Descripción	Prioridad
RF-01-01	El sistema permite al administrador gestionar (crear, modificar, cancelar) reservas de vehículos.	10
RF-02-01	El sistema se integra con 5 plataformas de pago en línea para procesar transacciones.	10
RNF-10-01	La interfaz de usuario es responsiva, accesible desde navegadores web modernos en escritorio y móviles.	10
RNF-12-01	La comunicación de datos y acceso remoto se realiza a través de internet mediante protocolos seguros (HTTPS/SSL).	10
RF-01-02	El sistema centraliza y almacena en base de datos la información de clientes, vehículos y transacciones.	9
RF-03-01	El sistema recibe datos en tiempo real de los dispositivos GPS instalados en la flota.	9

RNF-11-01	El consumo de recursos en memoria del servidor debe ser ligero y eficiente.	9
RF-03-02	El sistema permite al administrador visualizar la ubicación de los vehículos en un mapa.	8
RF-04-01	El sistema genera alertas y lleva el control de mantenimiento de la flota de vehículos.	8
RNF-13-01	El sistema es autónomo en su funcionamiento principal y no depende exclusivamente de sistemas externos para crear reservas.	8

Tabla 4. Requerimientos de prioridad mayor o igual a 8.

3.2. Metas y restricciones arquitectónicas según requerimientos

Basados en la priorización expuesta en la sección anterior y en la naturaleza del negocio de Autos Marbella, se plantean las siguientes metas y restricciones a cumplir con la arquitectura propuesta:

Accesibilidad y Diseño Responsivo: ya que los administradores y clientes interactuarán con la plataforma desde diferentes contextos, la arquitectura debe soportar una interfaz web intuitiva que se adapte perfectamente tanto a computadoras de escritorio como a dispositivos móviles.

Eficiencia de Recursos (Restricción de Memoria): el sistema debe estar diseñado bajo estándares de optimización de código y consultas a base de datos, garantizando que el consumo de recursos (CPU y memoria) en los servidores de despliegue sea mínimo y ligero.

Integración e Interoperabilidad: aunque Accelerilate es un sistema autónomo, la arquitectura debe ser lo suficientemente flexible (mediante el uso de APIs o microservicios) para permitir la comunicación bidireccional segura con hasta 5 plataformas de pagos, sistemas de facturación externos y APIs de rastreo GPS.

Seguridad de la Información: al manejar transacciones financieras, historial de clientes y ubicación de vehículos en tiempo real, es obligatorio implementar protocolos de comunicación seguros (como TLS/HTTPS) para la sincronización de datos y el acceso remoto a la plataforma.

Gestión Centralizada de Datos: la arquitectura debe soportar un motor de base de datos relacional altamente estructurado que garantice la integridad referencial entre el inventario de la flota, el registro de clientes y el historial de rentas/mantenimiento.

4. Vista de Casos de Uso

Un caso de uso es una secuencia general de eventos que describe todas las posibles acciones entre los actores y el sistema para una determinada funcionalidad. En esta sección se describen los Casos de Uso del sistema **Accelerilate**, abarcando las funcionalidades que permiten automatizar la gestión de renta de Autos Marbella.

En este modelo, interactúan actores humanos (como el **Administrador** y el **Cliente**) y actores de sistema (como el **Sistema GPS**, **Pasarelas de Pago** y **Sistema de Facturación**).

Las funcionalidades principales incluyen:

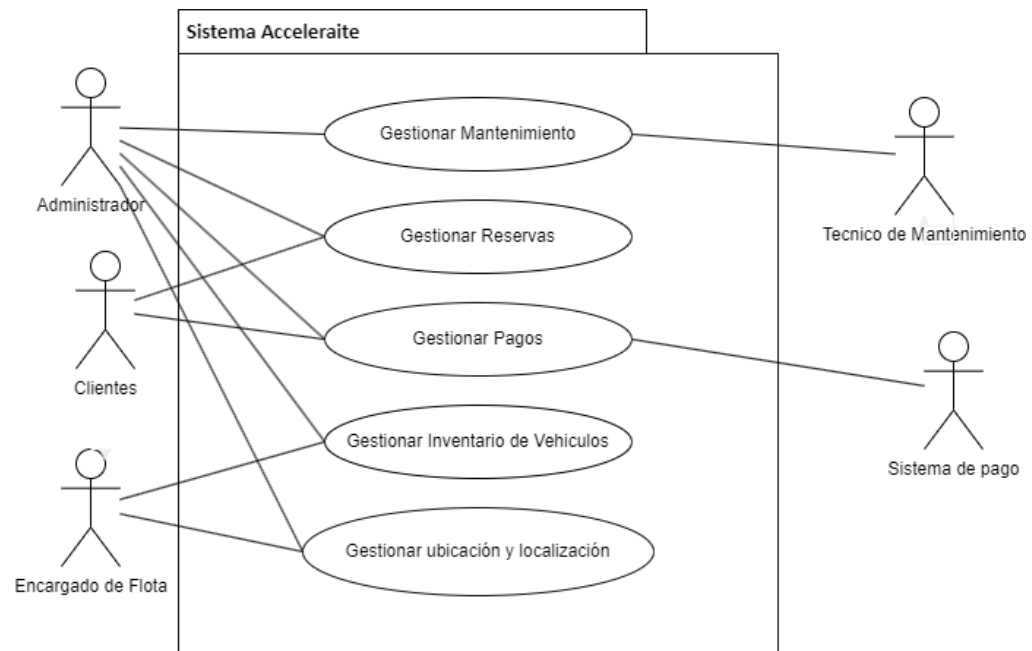
Gestión de Reservas: creación, modificación y cancelación de rentas de vehículos.

Monitoreo de Flota: visualización de la ubicación en tiempo real mediante la interfaz con los dispositivos GPS.

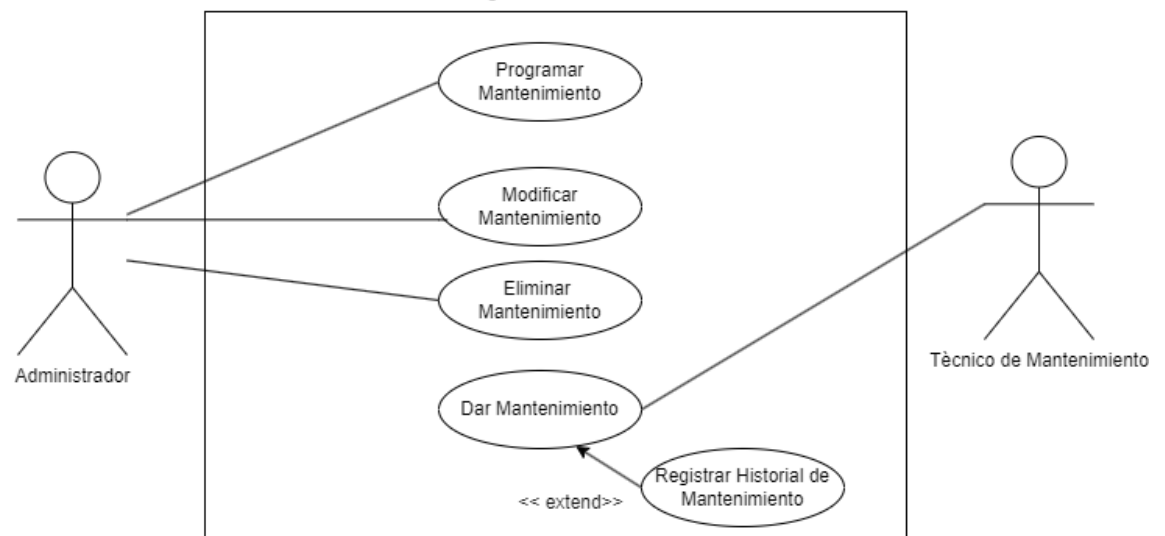
Procesamiento de Pagos: ejecución de cobros interactuando con las plataformas de pago integradas.

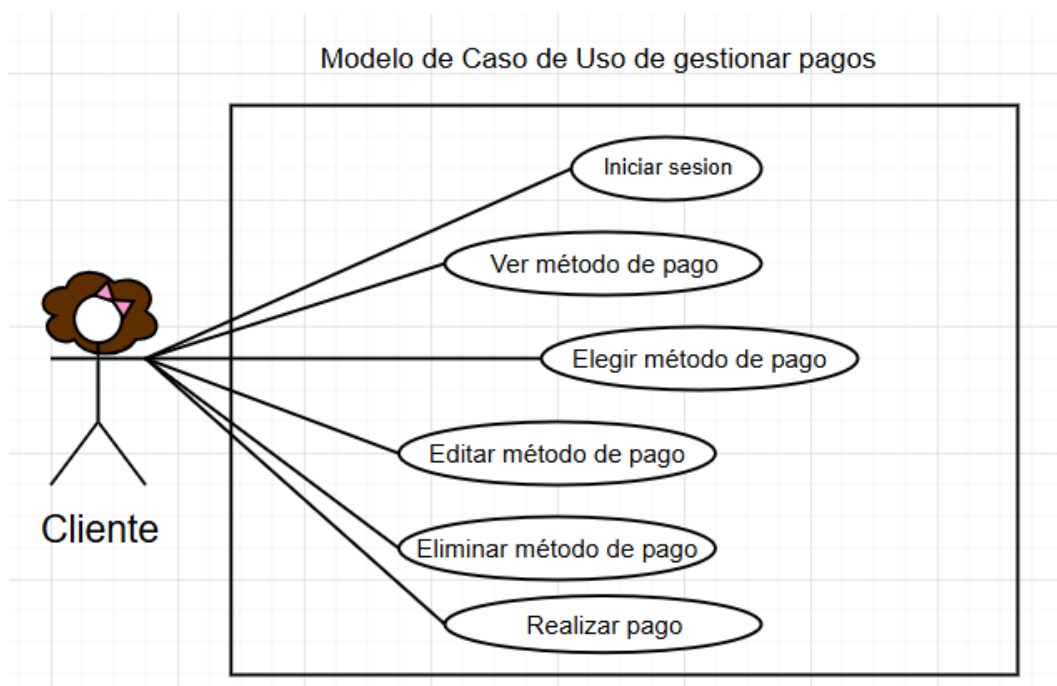
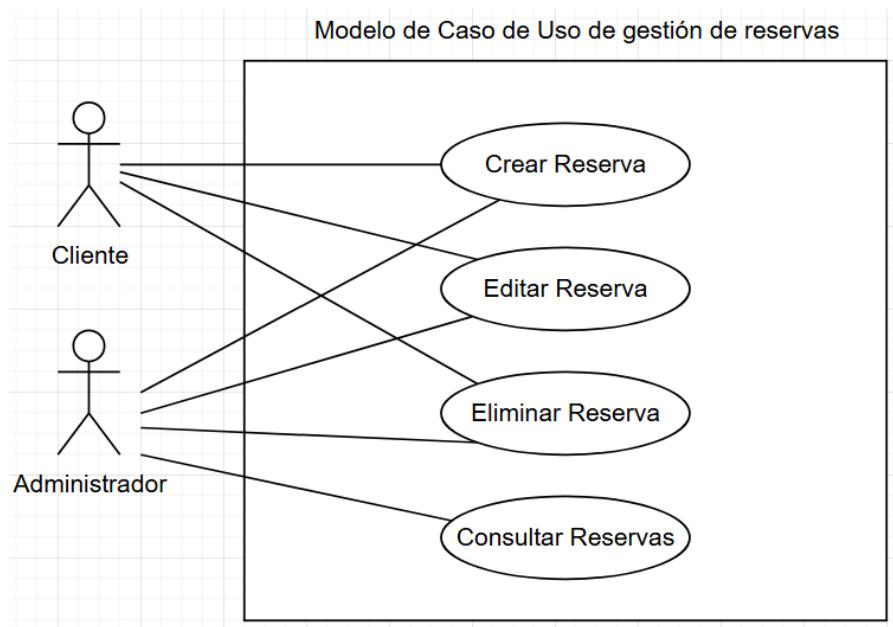
Control de Mantenimiento: actualización de estados del vehículo en el inventario para asegurar la disponibilidad.

Incluimos los siguientes diagramas de casos de uso:

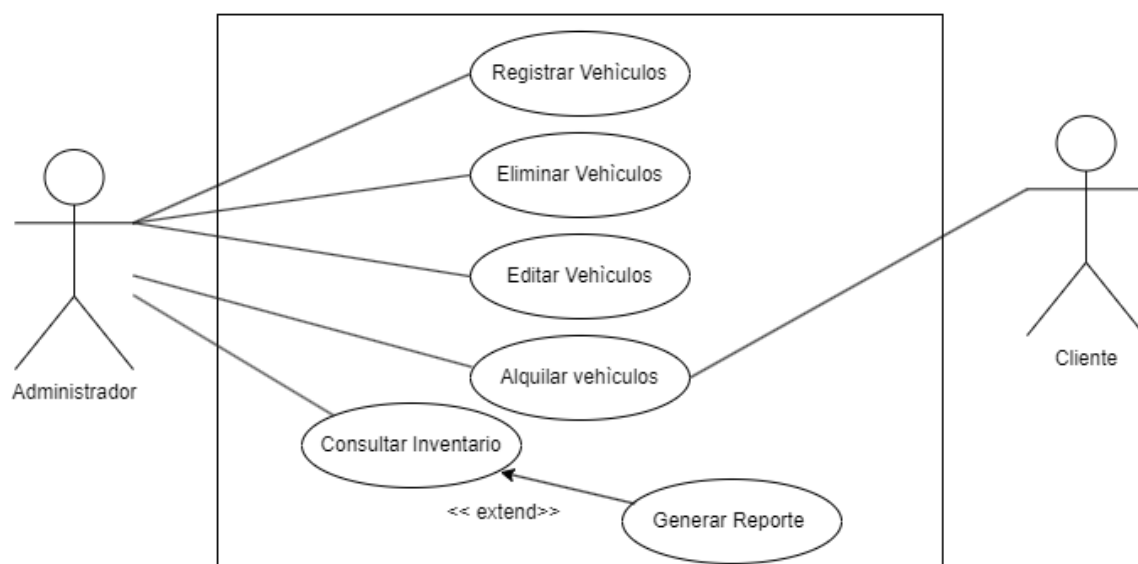


Modelo de caso de uso de gestión de Mantenimiento

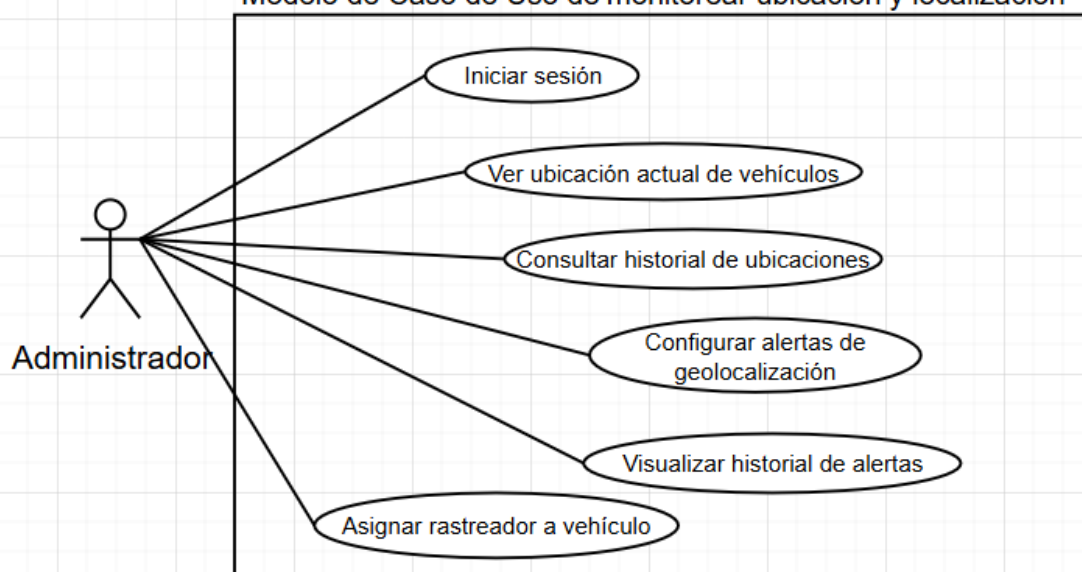




Modelo de caso de uso gestión inventario vehiculos



Modelo de Caso de Uso de monitorear ubicación y localización



La Tabla 5 muestra un resumen de los Casos de Uso principales del sistema Acceleraite para la gestión de Autos Marbella.

Tabla 5. Casos de Uso del sistema Acceleraite

Id	Nombre	Descripción
----	--------	-------------

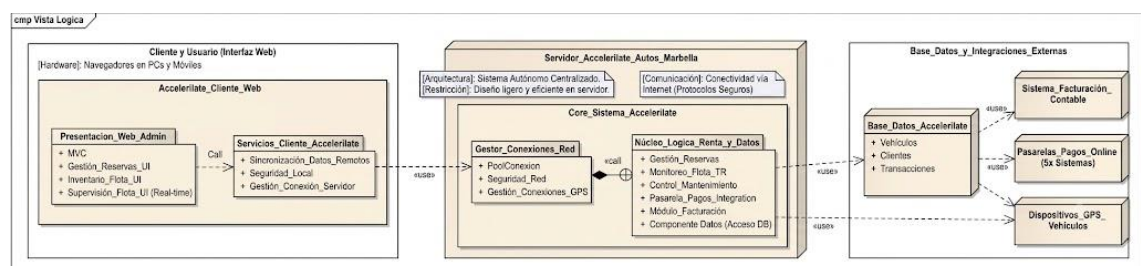
CU - 1	Gestionar Usuarios	Gestionar la información de los usuarios (administradores y clientes) existentes en el sistema.
CU - 1.1	Crear Usuarios	Crear cuentas de clientes o personal administrativo con sus datos requeridos.
CU - 1.2	Consultar Usuarios	Consultar información y el historial de rentas de los usuarios registrados.
CU - 1.3	Actualizar Usuarios	Modificar datos personales, de contacto o credenciales de los usuarios.
CU - 1.4	Desactivar Usuarios	Desactivar una cuenta para restringir el acceso al sistema.
CU - 2	Gestionar Flota (Vehículos)	Gestionar el inventario de vehículos disponibles para la renta.
CU - 2.1	Registrar Vehículo	Ingresar un nuevo vehículo al sistema con sus especificaciones, placas y dispositivo GPS asociado.
CU - 2.2	Actualizar Estado de Vehículo	Cambiar el estado del vehículo (disponible, rentado, en mantenimiento).
CU - 3	Gestionar Reservas	Administrar el ciclo de vida completo de la renta de un vehículo.
CU - 3.1	Crear Reserva	Asignar un vehículo a un cliente por un periodo de tiempo determinado, calculando el costo total.
CU - 3.2	Cancelar Reserva	Anular una reserva previamente creada liberando el vehículo en el inventario.

CU - 4	Procesar Pagos	Interactuar con las 5 plataformas de pago en línea para procesar el cobro de la renta.
CU - 5	Monitorear Flota (GPS)	Recibir y consultar la ubicación en tiempo real de los vehículos rentados a través de la integración GPS.
CU - 6	Control de Mantenimiento	Generar alertas y registrar las entradas al taller mecánico de los vehículos de la flota.
CU - 7	Entrar al Sistema (Login)	Ingresar al sistema mediante credenciales seguras y ver las funcionalidades según el rol del usuario.
CU - 8	Generar Reportes	Consultar datos estadísticos sobre ingresos, vehículos más rentados y estado general de la flota.
CU - 9	Salir del Sistema	Cerrar la sesión de forma segura guardando cualquier cambio pendiente.

5. Vista Lógica

La vista lógica se encarga de representar los requerimientos funcionales del sistema.

Esta sección describe las partes del diseño del modelo significativas para la arquitectura, tales como subsistemas y paquetes.



5.1. Visión General

La arquitectura general escogida para el sistema de Acceleraite es Cliente-Servidor en Capas, diseñada para ser desplegada en una infraestructura en la nube (como AWS o Azure).

En esta vista se observan los dos módulos principales: Cliente (Frontend) y Servidor (Backend). El cliente se comporta como una *Single Page Application* (SPA) ligera que se ejecuta en el navegador (computadoras o móviles), conteniendo las capas de presentación y validación de interfaz.

El servidor se encarga fundamentalmente de la lógica de negocio profunda, la orquestación de integraciones externas (GPS y pasarelas de pago), el manejo de la seguridad (autenticación/autorización) y la comunicación estructurada desde y hacia la Base de Datos del sistema.

5.2. Diseño Arquitectónico de Paquetes Importantes

5.2.1. Paquete Presentación

Este paquete está modelado bajo una arquitectura orientada a componentes, utilizando tecnologías modernas como React combinadas con librerías de diseño como Ant Design para asegurar una interfaz responsiva e intuitiva. Es el encargado de desplegar los formularios de reserva, los mapas de monitoreo de flota y los paneles de control (dashboards) mostrados a los administradores y clientes.

5.2.2. Lógica del Negocio

La lógica de negocio está construida sobre un framework robusto como Spring Boot en Java o un entorno basado en Node.js. Contiene toda la algoritmia necesaria para modelar las reglas de Autos Marbella: cálculo de tarifas de renta, validación de disponibilidad de inventario, procesamiento de transacciones financieras y la interpretación de las coordenadas enviadas por los dispositivos GPS.

5.2.3. Conexión

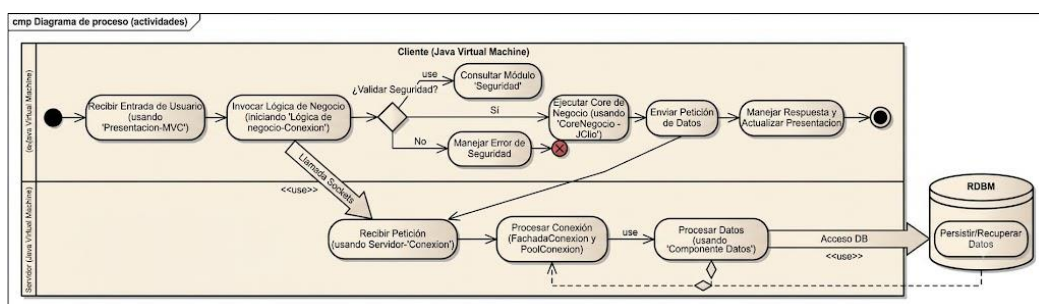
El paquete de conexión expone una API RESTful que actúa como puente de comunicación. No solo maneja la concurrencia de peticiones seguras (vía HTTPS) entre múltiples clientes web y el servidor, sino que también gestiona los *endpoints* externos que se comunican de forma asíncrona con los 5 sistemas de pago y el software de facturación.

5.2.4. Datos

Este paquete es el encargado de controlar la información que ingresa y sale de la Base de Datos relacional (por ejemplo, MySQL). Garantiza la integridad transaccional (ACID) de las reservas, asegurando que un mismo vehículo no pueda ser rentado por dos clientes en las mismas fechas. Sustenta el almacenamiento del inventario, los usuarios y el historial contable.

6. Vista de Procesos

En esta vista se realiza una descomposición en procesos ejecutables, tomando en cuenta principalmente los requerimientos no funcionales de desempeño, disponibilidad, concurrencia de múltiples sucursales/clientes, y la gestión eficiente de memoria en el servidor.



En la infraestructura en la nube de Accelerate, el despliegue estará orquestado utilizando contenedores (como Docker) para garantizar el aislamiento y el rendimiento. Estarán ejecutándose constantemente los siguientes procesos principales en el servidor:

Proceso del Servidor de Aplicaciones: un contenedor corriendo el backend Spring

Boot, que escucha las peticiones HTTP/REST entrantes.

Proceso del Motor de Base de Datos: el servicio del gestor relacional (MySQL)

procesando las consultas y escrituras.

Proceso de Escucha IoT (GPS): un subproceso en segundo plano dedicado

exclusivamente a recibir y normalizar las tramas de datos de ubicación enviadas por la flota de vehículos en tiempo real.

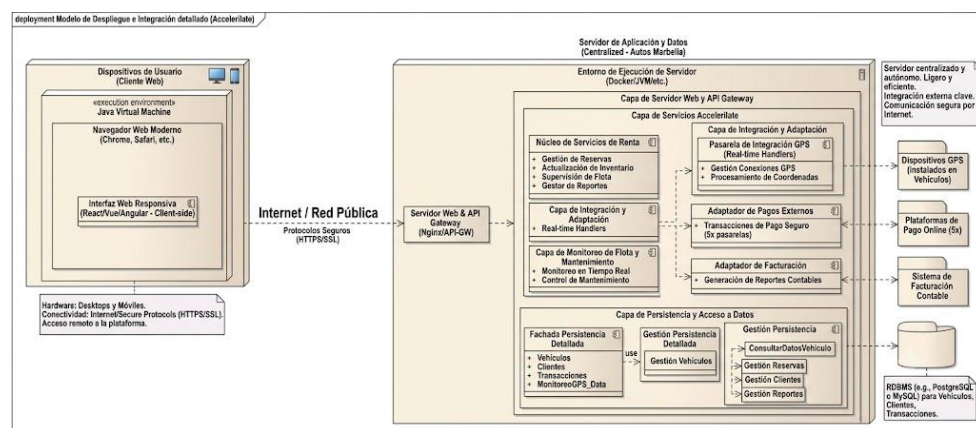
Por el lado del usuario, cada cliente del sistema Accelerate (administrador o usuario final) correrá un proceso de ejecución local dentro de su navegador web (la renderización de la interfaz en React).

La cantidad final de procesos de interfaz en ejecución será N, donde N es igual a la cantidad de usuarios conectados concurrentemente administrando reservas o monitoreando la flota, comunicándose todos de forma asíncrona con el proceso centralizado del servidor.

7. Vista de Despliegue

La configuración física de la red es muy simple ya que la arquitectura Cliente-Servidor permite que sea de esta manera.

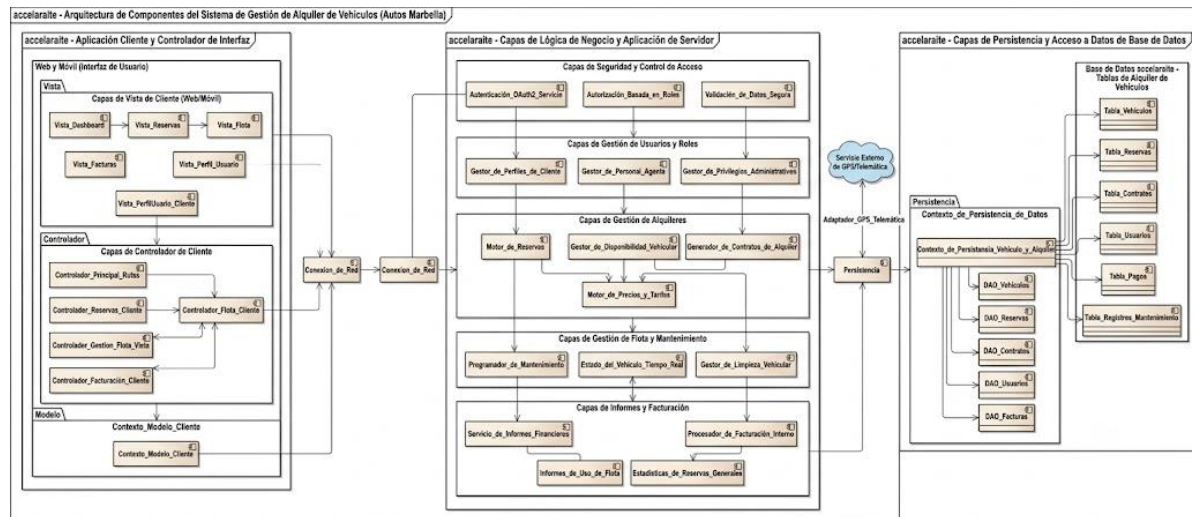
Como se puede ver en la siguiente ilustración:



Todos los clientes se conectarán directamente al servidor para intercambiar información con este, y poder funcionar de manera adecuada.

8. Vista de Implementación

En esta vista se detalla la manera como fue implementado el sistema Accelerilate, se describe visualmente las capas que tiene el sistema, cómo están distribuidas y sus principales funciones. El diagrama de Implementación se puede observar en la siguiente ilustración:



8.1. Visión General

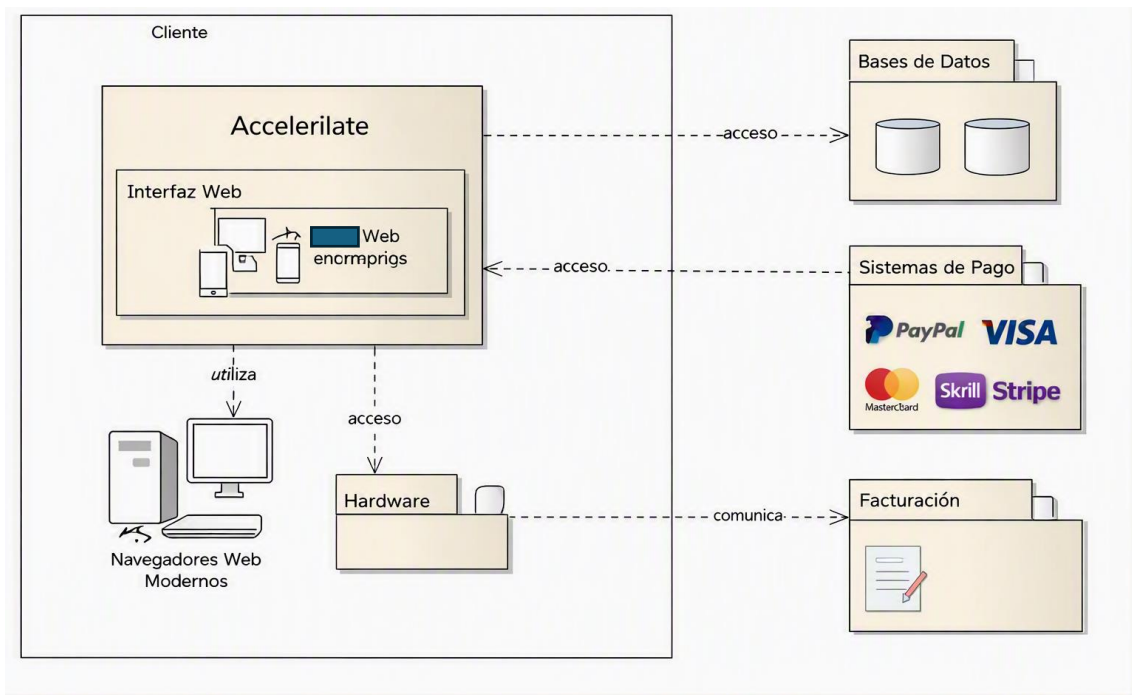
La implementación del sistema Accelerilate fue desarrollada bajo la arquitectura Cliente-Servidor en Capas (ver sección 2.2 Representación arquitectónica), aprovechando un entorno web en la nube donde se separa claramente el frontend (cliente web responsivo) del backend (servidor centralizado y API).

8.2. Capas

El sistema Accelerilate fue implementado en cuatro capas lógicas. Cada una de estas se describe a continuación:

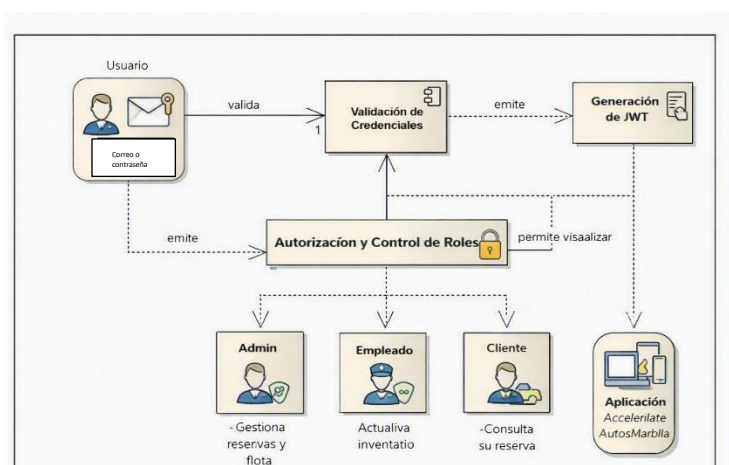
8.2.1. Presentación

Esta capa es la encargada de mostrar la información (dashboards, inventario de flota, mapas de GPS) asociada a cada tipo de usuario (Administrador, Cliente). Su función es recibir las acciones de los usuarios a través de navegadores web (de escritorio y móviles) y enviar las peticiones correspondientes a la capa de lógica mediante una API REST.



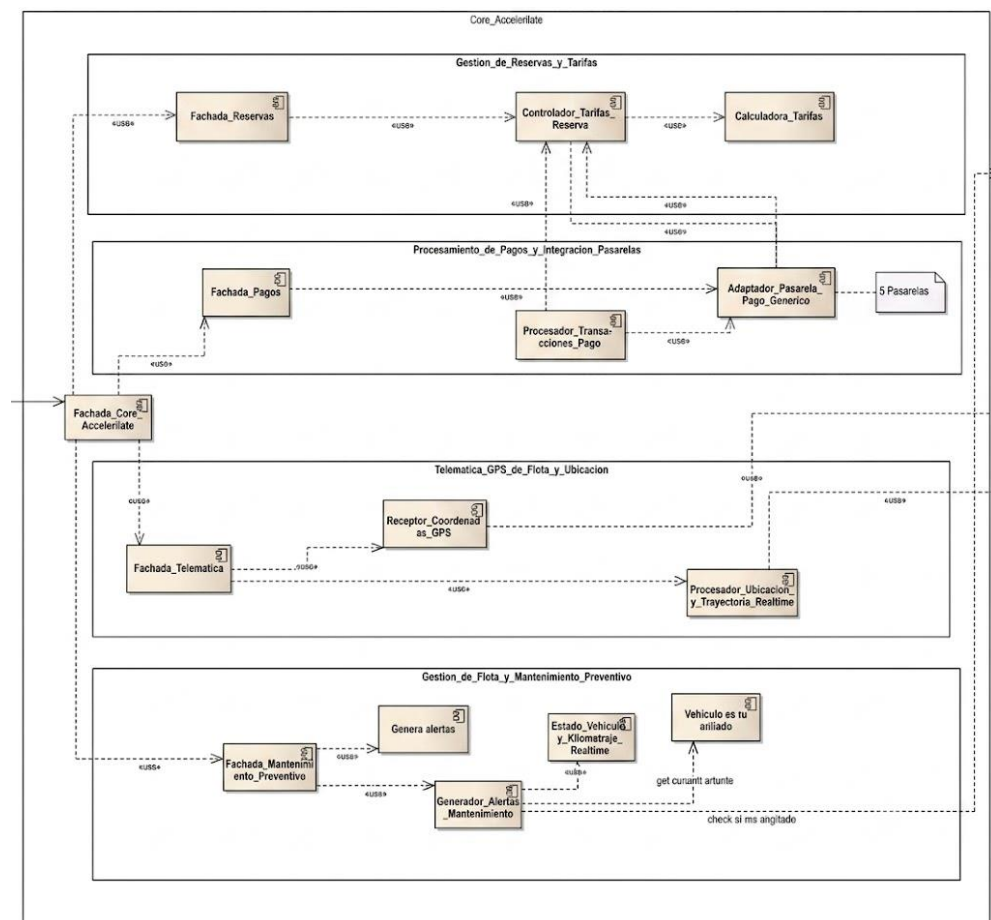
8.2.2. Seguridad

Esta capa es la encargada de validar que las credenciales (correo electrónico y contraseña) solicitadas por el usuario sean correctas. Además, gestiona la autorización (por ejemplo, mediante tokens JWT) para asegurar que se envíe y permita visualizar únicamente la información asociada a cada usuario según su rol (ver la sección Perfil de Usuarios en el Documento de Visión). En la siguiente ilustración se puede observar la capa de seguridad:



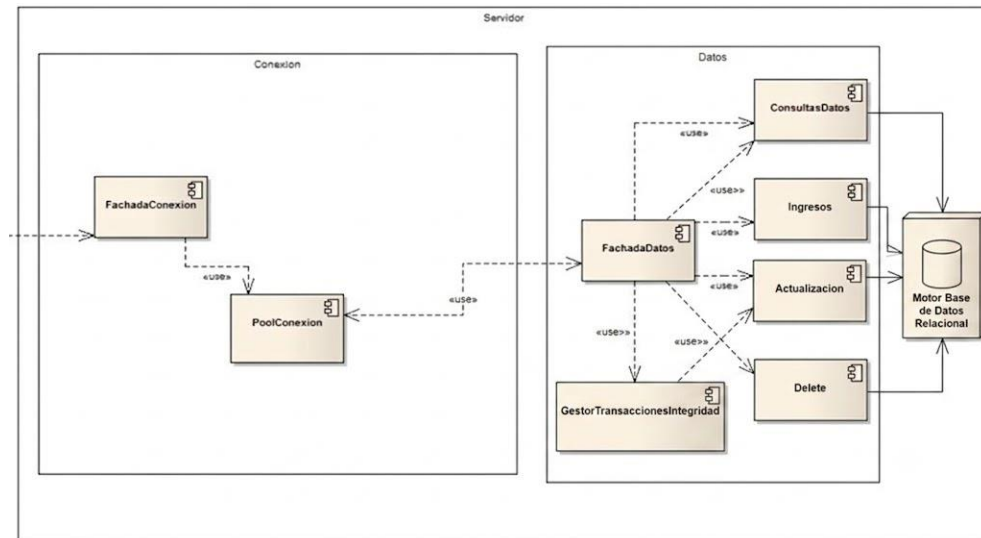
8.2.3. Lógica de Negocio (Core)

Esta capa es la encargada de administrar el comportamiento central y las reglas de negocio de Accelerilate. Responde a las peticiones realizadas desde la capa de presentación y procesa de forma autónoma operaciones críticas como: el cálculo de tarifas de reserva, la integración con las 5 pasarelas de pago, la recepción de coordenadas de los dispositivos GPS y la generación de alertas de mantenimiento de la flota.



8.2.4. Datos

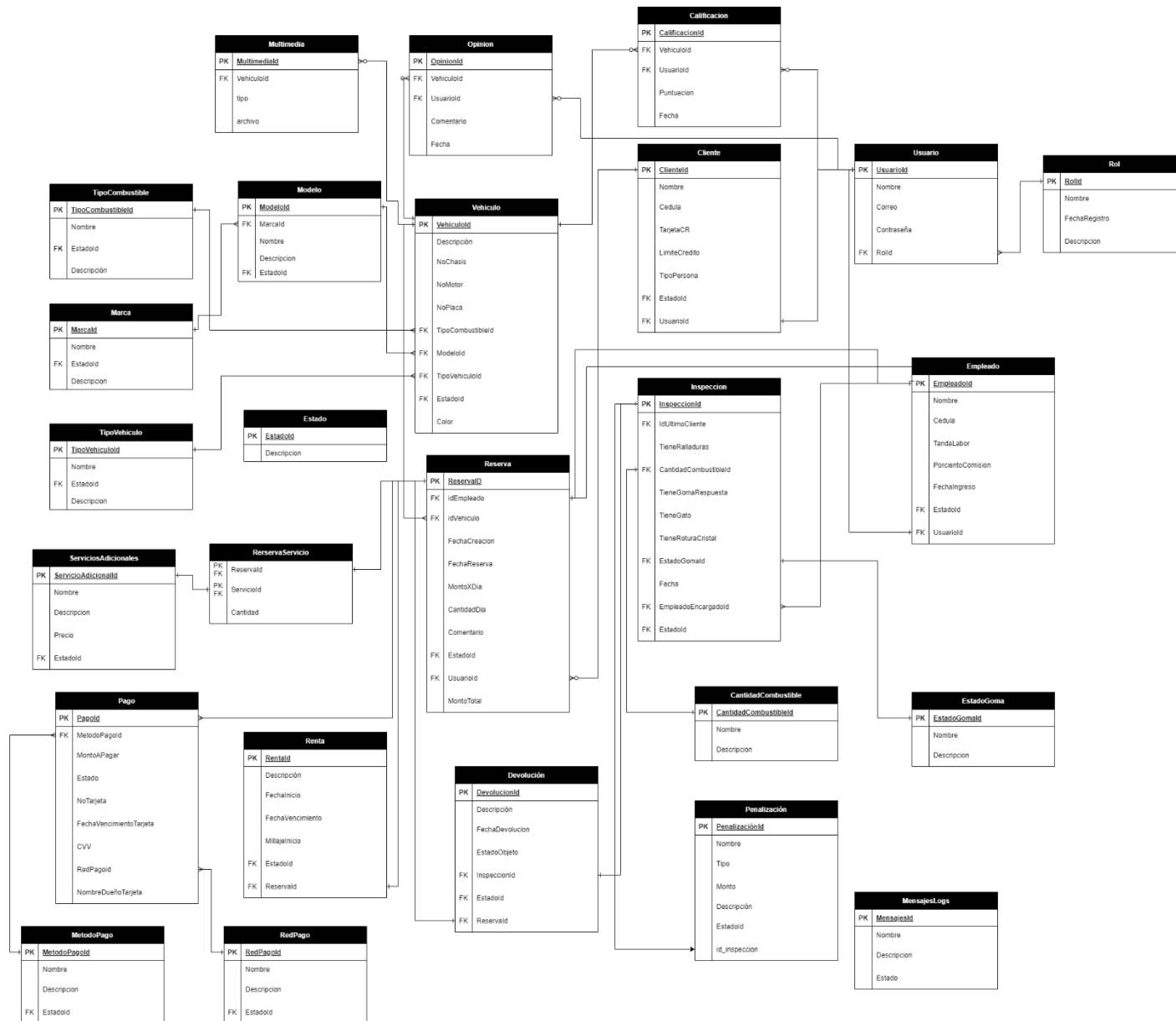
En esta capa se realiza el procesamiento de datos desde y hacia el motor de base de datos relacional para que la capa de lógica de negocio pueda funcionar correctamente y con integridad. Se encarga de gestionar conexiones eficientes y asegurar la persistencia transaccional (ej. que un vehículo no se rente dos veces en la misma fecha).



9. Vista de Datos

En esta vista se explica la manera como fueron almacenados los datos en la base de datos de Accelerilate. Esta vista es vital porque el sistema centraliza la gestión de usuarios, el inventario de vehículos, el historial de reservas, las transacciones financieras y los registros de ubicación (GPS).

En el diagrama de entidad-relación que se puede observar las tablas que se utilizaron para la persistencia de los datos garantizando la integridad referencial. En esta ilustración:



10. Tamaño y Desempeño

En esta sección se describen de manera general las características del software que impactan la arquitectura y las restricciones de desempeño.

10.1. Concurrencia de usuarios

El sistema Accelerilate está diseñado para permitir el acceso concurrente de múltiples administradores de Autos Marbella gestionando reservas, junto con los clientes realizando consultas, y el flujo constante de datos entrantes desde los dispositivos GPS de los vehículos.

Para cumplir con la restricción de diseño de ser ligero y eficiente minimizando el consumo de recursos de memoria, el sistema cuenta con un pool de conexiones a la base de datos y un manejador de concurrencia. Esto garantiza que el sistema no baje su desempeño en los servidores de despliegue, incluso en picos de alta demanda transaccional.

11. Calidad

En esta sección se describe cómo la arquitectura contribuye a las características no funcionales del sistema Accelerilate.

11.1. Seguridad

11.1.1. Despliegue de la información

El sistema Accelerilate controla el despliegue de la información de acuerdo al tipo de usuario. Por ejemplo, los administradores tienen acceso a la ubicación de toda la flota, reportes financieros y gestión de inventario, mientras que un cliente final solo puede ver el estado de sus reservas personales. La capa de seguridad es la encargada de validar estos privilegios antes de retornar cualquier dato confidencial.

11.1.2. Validación de la información del usuario

El sistema asegura la privacidad de la información comercial de Autos Marbella y los datos personales de los clientes. Como el sistema se comunica a través de internet, se utilizan protocolos seguros (HTTPS/SSL) y contraseñas encriptadas. La capa de seguridad valida cada inicio de sesión e intercepta cualquier petición malintencionada que intente modificar reservas o pagos sin autorización.

11.2. Usabilidad

11.2.1. Interfaz intuitiva y mapas interactivos

A diferencia de sistemas rígidos, Accelerilate cuenta con una interfaz web responsiva e intuitiva. Integra elementos visuales dinámicos, como mapas interactivos para el rastreo GPS en tiempo real y paneles de estado con códigos de colores para identificar rápidamente si un vehículo está rentado, disponible o en mantenimiento. Esto agiliza las operaciones diarias del administrador.

11.2.2. Ayudas tipo ToolTipText

El sistema Accelerilate contiene ayudas visuales tipo ToolTipText en cada uno de sus botones de acción (ej. "Aprobar Pago", "Cancelar Reserva", "Ver Ruta GPS"), así como validación clara de formularios, lo que disminuye la curva de aprendizaje del personal de Autos Marbella y mejora significativamente la usabilidad general del sistema.